

AI Traffic Sign Recognition & Driver Safety System

Hackathon Blueprint • MVP-first • Vibe-coded development

Core idea: Build an AI-powered road safety assistant that detects traffic signs and signals from a camera feed, explains them to the driver in real time, and combines road-context information with driver-alertness monitoring.

1. The Idea in Simple Terms

The project is a smart driving assistant. A camera observes the road and an AI vision model identifies traffic signs such as speed limits, stop signs, no-entry signs, pedestrian crossings, turns, warnings, and traffic signals. The system immediately tells the driver what it detected and what it means.

For the hackathon, the product should be presented as a **working safety-assistance prototype**, not as a fully autonomous driving system. The goal is to demonstrate fast visual recognition, useful road context, and driver-awareness alerts in one connected experience.

The broader dashboard can show a map of the route with traffic conditions, known traffic signals/signs, construction areas, speed limits, and nearby garages. A separate camera pipeline can estimate whether the driver is attentive or showing signs of drowsiness. A voice layer can announce important events without requiring the driver to look at a screen.

2. Problem Statement

- Drivers can miss or notice traffic signs too late, especially in unfamiliar roads or poor visibility.
- A sign detector alone does not provide useful road context such as the current speed limit, construction, traffic, or nearby assistance.
- Driver fatigue can reduce reaction time and attention.
- Most hackathon prototypes solve only one part of the problem. This project connects road perception, contextual mapping, voice alerts, and driver monitoring into one safety-oriented system.

3. Proposed Solution

Input → AI perception → decision layer → alert → map/dashboard

- **Camera:** receives live video or uploaded road footage.
- **Traffic-sign model:** detects and classifies signs and traffic lights.
- **Context engine:** combines detections with GPS/map information.
- **Driver-monitoring model:** estimates visible signs of inattention or drowsiness.
- **Alert engine:** prioritizes important events and generates text/voice warnings.
- **Web dashboard:** displays the camera feed, detections, route, road context, and driver state.

4. MVP Scope for the Hackathon

Do not attempt to build every possible feature at production quality. The MVP should prove the core loop end-to-end.

Feature	MVP implementation	Priority
Traffic sign detection	YOLO-based detection on live webcam/video	Must have

Feature	MVP implementation	Priority
Traffic light detection	Classify red/yellow/green where supported by dataset/model	Must have
Real-time alert	On-screen alert + text-to-speech for important signs	Must have
Driver monitoring	Face/eye landmarks or webcam-based drowsiness heuristic	Must have
Map	Interactive route/map with selected road context	Must have
Speed limit	Demo location data or map/API-derived information	Should have
Construction/traffic	Map/API layer or curated demo dataset	Should have
Nearby garages	Map search/POI layer	Should have
Cloud-scale deployment	Optional; local demo is acceptable	Nice to have

5. Recommended System Architecture

A practical architecture for a vibe-coded hackathon project is to keep the frontend, AI inference service, and map/context service separate. This makes debugging much easier.

Layer	Recommended technology	Responsibility
Frontend	React + Vite + Tailwind CSS	Dashboard, camera UI, map, alerts, status cards
AI backend	Python + FastAPI	Inference API and AI pipeline
Object detection	Ultralytics YOLO	Traffic signs/signals detection
Computer vision	OpenCV	Frames, preprocessing, camera/video handling
Driver monitoring	MediaPipe Face Landmarker / OpenCV	Face/eye/head-pose features
Maps	Leaflet/MapLibre + OpenStreetMap	Interactive map and route visualization
Data/API layer	FastAPI + JSON/SQLite	Road-context and demo data
Voice	Browser SpeechSynthesis or Python TTS	Read important alerts aloud
Deployment	Vercel for frontend; Render/Railway/local for API	Hackathon hosting
Version control	Git + GitHub	Team collaboration and rollback

6. Tech Stack — What Each Part Does

Python: AI and computer-vision backend language.

Ultralytics YOLO: object detection model framework. Start with a pretrained model and fine-tune only if the dataset/time justify it.

OpenCV: captures frames, resizes images, draws bounding boxes, and handles video.

FastAPI: exposes endpoints such as /detect, /driver-status, and /health.

MediaPipe: provides face/landmark signals useful for estimating eye closure and head orientation.

React + Vite: fast web dashboard development.

Tailwind CSS: rapid UI styling.

Leaflet/MapLibre + OpenStreetMap: map visualization without needing to build map rendering yourself.

SQLite/JSON: simple local storage for curated road/sign/garage demo data.

GitHub: shared repository and version history.

7. Step-by-Step Build Guide

Step 1 — Create the repository

- Create a GitHub repository and a clear folder structure.
- Suggested structure: frontend/, backend/, models/, data/, scripts/, docs/.
- Create .gitignore with venv/, __pycache__/, *.pyc, .env, node_modules/.

Step 2 — Set up the Python environment

- Create a virtual environment: `python -m venv venv`
- Activate it on Windows: `venv\Scripts\activate`
- Install core packages: `ultralytics`, `opencv-python`, `fastapi`, `uvicorn`, `mediapipe`, `numpy`, `python-multipart`.

Step 3 — Build traffic-sign detection first

- Start with a pretrained YOLO model to validate the pipeline.
- Test it on images and recorded road videos before attempting live webcam inference.
- If pretrained performance is insufficient, use a traffic-sign dataset and fine-tune a small YOLO model.
- Track class name, confidence, bounding box, timestamp, and frame ID.

Step 4 — Add the FastAPI inference service

- Create `/health` to verify the backend.
- Create `/detect` accepting an image/frame and returning detections as JSON.
- Return a consistent schema: `{label, confidence, x1, y1, x2, y2}`.
- Keep inference code separate from API routes so it can be tested independently.

Step 5 — Build the frontend dashboard

- Create a camera/video panel with detection overlays.
- Add a detection history panel showing the last few recognized signs.
- Add a driver-status card: ATTENTIVE / WARNING / DROWSINESS DETECTED.
- Add a map panel and an alert banner.

Step 6 — Add voice alerts

- Only speak high-value events to avoid distracting or repeatedly talking to the driver.
- Example: 'Speed limit 40 kilometres per hour detected.'
- Add a cooldown so the same sign is not announced every frame.

Step 7 — Add driver monitoring

- Use the webcam to locate the driver's face.
- Estimate eye closure using facial landmarks or an eye-aspect-ratio-style metric.
- Combine sustained eye closure, long blink duration, and head orientation into a simple warning score.
- Use states such as Normal → Attention Warning → Drowsiness Warning.
- Present this as an assistance feature, not a medical diagnosis or proof that someone is sleepy.

Step 8 — Add the map

- Display the current/demo route.

- Add markers for traffic signals, traffic signs, construction zones, and garages.
- Show speed-limit information for the selected road segment where reliable data is available.
- For a hackathon, a curated demo dataset is acceptable if clearly labelled as demo/simulated data.

Step 9 — Connect everything

- Frontend sends frames or receives detections from the backend.
- Backend returns detection results and driver-monitoring status.
- Frontend updates the map and alert system based on location/context.
- Use one event format across the application so every component understands the same detection.

Step 10 — Test the complete demo

- Test day/night or different lighting conditions if possible.
- Test false detections, low confidence, repeated signs, camera failure, and backend disconnection.
- Measure approximate FPS, detection latency, and recognition confidence.

8. Suggested API Design

Endpoint	Purpose	Example response
GET /health	Check backend status	status: ok
POST /detect	Detect signs from a frame	detections: [...]
POST /driver-status	Estimate driver state	state: attentive
GET /road-context	Return road/map context	speed_limit, traffic, construction
GET /garages	Return nearby garages	locations: [...]

9. Team Split for a 4-Person Hackathon Team

- **Person 1 — AI:** YOLO model, dataset, training/inference, evaluation.
- **Person 2 — Backend:** FastAPI, frame pipeline, API integration, driver-monitoring service.
- **Person 3 — Frontend/Map:** React dashboard, map, UI, alerts.
- **Person 4 — Integration/Demo:** connects modules, testing, deployment, demo dataset, presentation.

Everyone should use Git branches and pull requests. Avoid having all four people edit the same files at the same time.

10. Three-Day Hackathon Execution Plan

Day 1 — AI core + backend

- Set up repo, Python environment, frontend skeleton.
- Run YOLO on sample traffic-sign images/video.
- Create FastAPI /detect endpoint.
- Get driver-monitoring prototype working independently.
- Checkpoint: detection works and returns structured JSON.

Day 2 — Product integration

- Connect React dashboard to backend.
- Add bounding boxes, confidence labels, detection history.
- Add voice alerts and cooldown logic.
- Build map with road context, construction, garages, traffic and speed-limit demo data.
- Checkpoint: complete user journey works on one machine.

Day 3 — Polish + testing + pitch

- Improve UI and reliability.
- Test failure cases and latency.
- Prepare a 2–3 minute demo video/script.
- Prepare architecture diagram and explain what is real vs simulated.
- Deploy if stable; otherwise prioritize a reliable local demo.

11. Demo Flow for Judges

- 1. Open the dashboard and show the live camera.
- 2. Point the camera at a speed-limit sign: model detects it and displays confidence.
- 3. Voice announces the sign.
- 4. Show a traffic signal or warning sign and demonstrate the same loop.
- 5. Show the map: route, speed limit, construction/traffic layer and nearby garages.
- 6. Demonstrate driver-monitoring warning using a controlled test condition.
- 7. Open detection history / event log to show that events are recorded.
- 8. Finish by explaining that the system combines perception + context + driver awareness rather than only recognizing an image.

12. Accuracy, Speed & Evaluation

- **Precision/Recall:** evaluate how often detected signs are correct and how many relevant signs are missed.
- **Confidence threshold:** suppress low-confidence detections from voice alerts.
- **Latency:** measure time from frame capture to displayed alert.
- **FPS:** report approximate inference/display speed on your demo hardware.
- **False-alert rate:** manually test common objects that should not trigger sign alerts.
- For the hackathon pitch, report measured values from your actual demo rather than claiming guaranteed high accuracy.

13. Safety & Product Boundaries

- The prototype should be described as a **driver-assistance and awareness system**, not autonomous driving.
- Do not claim that a detected sign is always correct; use confidence and uncertainty handling.
- Do not claim driver monitoring can diagnose sleepiness, nausea, illness, or medical conditions.
- If the model is uncertain, show 'Uncertain detection' instead of inventing a result.
- Map information such as traffic, construction, speed limits, and garages can become outdated; show the source/time or label demo data.

14. Vibe-Coding Workflow

- Use an AI coding assistant to generate one module at a time rather than asking it to build the entire project in one prompt.
- Give the assistant the current folder structure, requirements, API contract, and definition of done.
- After every generated change, run the application and test it.
- Keep a README with setup commands and environment variables.
- Use small commits: 'add YOLO inference', 'add /detect API', 'connect camera UI', etc.
- Never paste secrets/API keys into prompts, GitHub, frontend code, or .env files committed to the repository.

15. Master Build Order

Camera → YOLO detection → FastAPI → React UI → voice alerts → driver monitoring → map/context → integration → testing → deployment → presentation.

16. Final Hackathon Positioning

The strongest way to present the project is not as 'another traffic-sign detector'. Present it as a **real-time road-awareness assistant**: it sees the road, identifies important signs and signals, explains them to the driver, adds location-based context, and monitors visible signs of driver inattention. The AI model is the core perception engine; the dashboard, map, alert system, and driver-monitoring layer turn that model into a demonstrable product.

17. Quick Checklist Before Submission

- ■ YOLO detection works on a real video/webcam.
- ■ Sign labels and confidence scores appear correctly.
- ■ Voice alert works and has a cooldown.
- ■ Driver-monitoring state changes correctly in a controlled test.
- ■ Map loads and markers/context are visible.
- ■ Backend health endpoint works.
- ■ Frontend handles backend failure gracefully.
- ■ No API keys/secrets are committed.
- ■ README contains setup instructions.
- ■ Demo flow has been rehearsed end-to-end.
- ■ Presentation clearly distinguishes live AI output from simulated/curated map data.

Bottom line: Build the smallest reliable end-to-end system first. A polished demo where detection → explanation → map context → driver alert works consistently is more valuable in a short hackathon than a large set of unfinished features.